

AI-Powered Job Recommendation System

Overview

This Streamlit application serves as the frontend interface for the AI-Powered Job Recommendation System. The application enables users to upload resumes, process them using AWS-powered backend services, retrieve AI-generated job recommendations, analyze skill gaps, and visualize job-match insights.

The system integrates:

- Streamlit
 - AWS S3
 - AWS Lambda
 - API Gateway
 - MongoDB
 - Amazon Bedrock
 - RAG-based job matching pipeline
-

Objectives

The application is designed to:

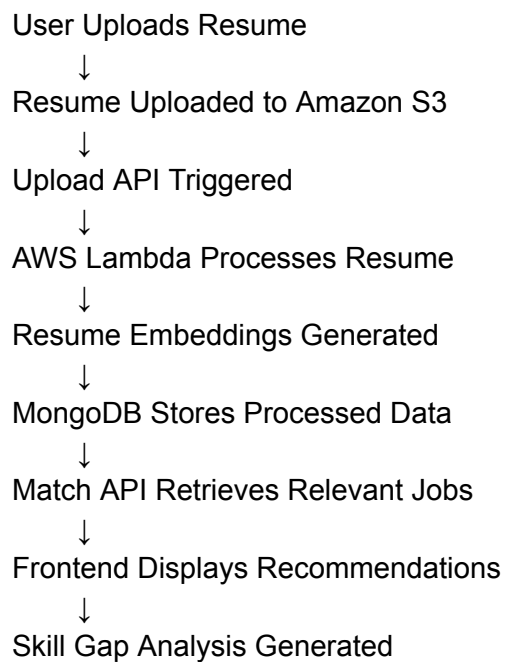
- Upload candidate resumes securely
 - Trigger backend AI processing
 - Retrieve semantic job recommendations
 - Display ranked job matches
 - Visualize candidate skill gaps
 - Recommend learning resources
 - Refresh latest jobs dynamically
-

Technologies Used

Technology	Purpose
Streamlit	Frontend UI

Boto3	AWS SDK for Python
Amazon S3	Resume storage
API Gateway	Backend API access
AWS Lambda	Resume processing
Pandas	Data handling
Matplotlib	Data visualization
Requests	REST API communication
dotenv	Environment variable Management

Application Workflow



Core Functionalities

1. Resume Upload

The application allows users to upload PDF resumes through Streamlit's file uploader.

Features

- PDF validation
- Unique resume ID generation
- Upload to Amazon S3
- Metadata tagging
- Duplicate upload prevention using session state

Upload Flow

```
uploaded_file = st.file_uploader(  
    "Upload Resume (PDF)",  
    type=["pdf"]  
)
```

2. Resume Processing

Once uploaded:

1. Resume is stored in S3
 2. Backend upload API is triggered
 3. Lambda extracts text
 4. AI embeddings are generated
 5. Resume summary is created
 6. Data stored in MongoDB
-

3. Session State Management

The application uses Streamlit session state to persist user data across reruns.

Stored Variables

Variable	Purpose
resume_id	Stores uploaded resume reference
matches_data	Stores job recommendation results

Example

```
if "resume_id" not in st.session_state:  
    st.session_state["resume_id"] = None
```

4. Job Matching Engine

Users can trigger AI-based job recommendations using the “Find Matching Jobs” button.

Backend Endpoint

POST /match

Request Payload

```
{  
  "resume_id": "uuid.pdf"  
}
```

Returned Data

- Job title
 - Company
 - Match score
 - AI reasoning
 - Recommended courses
 - Apply links
-

5. Ranked Job Recommendations

Matching jobs are displayed in ranked order based on similarity score.

Features

- Sort by match percentage
 - Clickable application links
 - Dynamic HTML rendering
-

6. Match Score Visualization

The system generates a bar chart showing match distribution across jobs.

Visualization

- X-axis: Job titles
- Y-axis: Match scores

Implemented using:

```
st.bar_chart(chart_df)
```

7. Skill Gap Analysis

The application performs keyword-based skill analysis from AI reasoning data.

Tracked Skills

- Python
- SQL
- AWS
- Docker
- Kubernetes
- Spark
- Airflow
- MongoDB
- Machine Learning
- APIs
- Linux
- Pandas
- ETL
- Data Engineering

Output

- Skill match percentages
 - Missing skills identification
 - Horizontal bar chart visualization
-

8. AI Reasoning Display

Each recommended job contains AI-generated reasoning explaining:

- Why the candidate matches
- Relevant skills identified
- Resume-job similarity analysis

Displayed using expandable sections.

9. Learning Recommendations

The system optionally displays recommended courses for missing skills.

Example

- AWS Certification Course
 - Docker Fundamentals
 - Machine Learning Basics
-

10. Daily Job Refresh

The sidebar includes functionality to refresh job postings from external APIs.

Endpoint

POST /refresh

Features

- Fetch latest jobs
 - Update MongoDB
 - Display insertion statistics
 - Timeout handling
 - Error handling
-

Conclusion

This Streamlit frontend acts as an intelligent interface for an AI-powered recruitment and recommendation platform. By integrating AWS cloud services, AI embeddings, semantic search, and visualization techniques, the application provides users with meaningful job recommendations and actionable skill insights.

The system demonstrates practical implementation of:

- Generative AI
- Retrieval-Augmented Generation (RAG)
- AWS Serverless Architecture
- Semantic Search
- Data Visualization
- AI-driven Career Intelligence